

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-a5e4b91cb6f1fc33d.jsonl`  
- SHA-256: `ef21c1e38c720966e62adf4c89bd90fe2d03a7b7f790870cb4de834887168ffa`  
- Source modified: `2026-06-16T06:35:03+00:00`  
- Imported at: `2026-07-05T16:48:29+00:00`  
- project: `wf_ef85c152-821`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T06:32:58.971Z)

You are a SKEPTICAL reviewer. Confirm or REFUTE this audit finding against the ACTUAL code in the worktree C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (cite file:line). Decide isMustFix = is this a REAL issue that must be fixed before relying on the suite, vs a non-issue / nice-to-have / already-handled? Default to 'refuted' or 'partial' (isMustFix=false) unless you can confirm it in the code.

FINDING [PRIVACY / de-attribution ? completeness of `\_scan\_forbidden`, surrogate non-reversibility, refusal-gate bypass, raw-trajectory fingerprinting, and honesty of the doc's bounded claims, for the shipped `freeze\_real\_fixture` de-attribution path. / high]:

`\_scan\_forbidden` does NOT catch free-text identifiers in `source\_note`/`label` ? they are committed verbatim into provenance.json

EVIDENCE: golden_fixtures.py:501-535 `\_scan\_forbidden` scans `text` only for (a) FORBIDDEN_KEYS biometric substrings, (b) a 32-hex MD5, (c) source-PATH tokens (sp/p.name/p.stem). `source\_note` (line 663) and `label` (line 643, default "cleared_real") are written verbatim into provenance.json and THEN scanned (lines 680-681), but none of the three categories matches a free-text identifier. Proven empirically: freeze_real_fixture(..., label='mens singles final', source_note='Lin Dan vs Lee, India Open 2024, Court 1') FROZE with NO refusal and committed both strings into provenance.json. The earlier-passing test test_golden_real_fixtures.py:155 only catches a leak because it passes `source\_note=str(traj)` ? a PATH token the scan does catch ? masking the free-text hole. The lens's question 'a caller-supplied identifying slug, an identifying source_note' is answered: source_note is NOT covered.

PROPOSED FIX: The scan cannot validate free text, so the contract must be enforced differently: (a) hard-restrict `source\_note`/`label` to an allow-list charset/length or a controlled vocabulary, or drop them from the committed provenance entirely (move any human note to the off-git surrogate map); and (b) add a test that a free-text identifying source_note/label is either rejected or never persisted. At minimum, document loudly in the CLI `--source-note`/`--label` help and in §4c that these fields are committed UNSCANNED and the operator is solely responsible ? the current help says 'NEVER a path/name/id' but nothing enforces it for names/ids.

2. assistant (2026-06-16T06:33:02.365Z)

I'll verify this audit finding against the actual code. Let me start by reading the relevant files.

3. user (2026-06-16T06:33:03.508Z)

```
480  
481  
482     def _new_surrogate_slug() -> str:  
483         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).  
484  
485         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a  
random,  
486         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
```

```

over a
487         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
488         """
489         return f"fx_{os.urandom(8).hex()}"
490
491
492     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
path tokens
493     #: that collide with these are NOT a leak ? exclude them so a short source stem like
``traj`` can't
494     #: false-positive against the literal
``trajectory.csv``/``labels.csv``/``expected.json``.
495     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
496         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
497         "provenance", "provenance.json",
498     )
499
500
501     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
502         """POSITIVE privacy assertion ($4c, red-team?): raise if ``text`` leaks anything
attributable.
503
504         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
505         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
506         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
507         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
508         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
509         """
510         low = text.lower()
511         for key in FORBIDDEN_KEYS:
512             if key in low:
513                 raise RefusalError(
514                     f"{where}: forbidden key substring {key!r} found in written output ? "
515                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
516                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md $4c).")
517         m = _MD5_RE.search(text)
518         if m:
519             raise RefusalError(
520                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
521                 f"video_id shape and an attribution back-channel; never write it ($4c/$5).")
522         for sp in source_paths:
523             if not sp:
524                 continue
525             p = Path(sp)
526             for token in (sp, p.name, p.stem):
527                 t = (token or "").strip().lower()
528                 # Skip a token that is itself a substring of a schema-literal the writers
always emit
529                 # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
530                 if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
531                     continue
532                 if t in low:
533                     raise RefusalError(
534                         f"{where}: source-path token {token!r} found in written output ? the
source "
535                         f"filename/path must never be persisted ($4c metadata strip).")

```

```

536
537
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
541         ``(Frame,Visibility,X,Y)`` rows.
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
543         invariant under a
544         pure time shift); only the absolute frame index moves toward 0, severing the match
545         timeline.
546         """
547         rows: List[Tuple[int, int, float, float]] = []
548         for p in points:
549             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
550             float(p.y)))
551         rows.sort(key=lambda r: r[0])
552         return rows
553
554     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
555         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
556         invariant)."""
557         return [(s - t0_sec, e - t0_sec) for s, e in gts]
558
559     def freeze_real_fixture(
560         label: str,
561         trajectory_csv_path: Any,
562         labels_csv_path: Any,
563         *,
564         license: str,
565         minors_free: bool,
566         owned: bool,
567         fps: float,
568         frame_width: float,
569         fixtures_dir: Path = FIXTURES_DIR,
570         slug: Optional[str] = None,
571         time_origin_reset: bool = True,
572         source_note: str = "",
573     ) -> Dict[str, Any]:
574         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
575         de-attributed
576         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
577         row P1.)
578
579         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
580         is emitted
581         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
582         blank string.
583         The committed bytes carry NO filename / video_id / match / player / capture-date ?
584         only an
585         opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
586         default), the
587         reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
588         assertion
589         re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
590         attributable
591         slipped in.
592
593         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
594         attributable
595         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
596         SRB). De-
597         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off

```

```

(P2) and the
586         owner-asserted Fork-A / minors / biometric-exclusion flags.
587
588         Returns the recomputed ``expected.json`` snapshot dict (identical to
589         ``replay_fixture``).
590         """
591         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
592         -----
593         reasons: List[str] = []
594         if minors_free is not True:
595             reasons.append("minors_free must be True (DPDP child = under 18; behavioural
596             monitoring "
597             "of a minor is prohibited ? §3, guardrail 'exclude minors')")
598         if owned is not True:
599             reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
600             commit; "
601             "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
602         if not (isinstance(license, str) and license.strip()):
603             reasons.append("license must be a non-blank string (per-record provenance tag ?
604             §4c "
605             "'provenance hard-fail gate')")
606         if reasons:
607             raise RefusalError(
608             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
609             does NOT "
610             "cure provenance/licensing; this gate sits behind counsel sign-off,
611             §6/P2):\n - "
612             + "\n - ".join(reasons))
613         traj_path = Path(str(traj_path))
614         labels_path = Path(str(labels_path))
615         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
616         name ---
617         out_slug = slug if slug is not None else _new_surrogate_slug()
618         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
619         ----
620         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
621         if not points:
622             raise ValueError(f"{traj_path}: no parseable trajectory rows
623             (Frame,Visibility,X,Y)")
624         gts = load_gt_intervals(str(labels_path), strict=True)
625         # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
626         -----
627         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
628         because
629         # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
630         end|), the
631         # score is bit-identical with vs without reset ? only the absolute timeline is
632         severed. The
633         # quality-floor / characterization snapshot is therefore unchanged (asserted in the
634         tests).
635         if time_origin_reset:
636             t0_frame = min(int(p.frame) for p in points)
637             t0_sec = t0_frame / float(fps)
638         else:
639             t0_frame, t0_sec = 0, 0.0
640         rows = _reset_trajectory_rows(points, t0_frame)
641         reset_gts = _reset_labels(gts, t0_sec)
642         # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
643         -----
644         fdir = fixtures_dir / out_slug

```

```

634     fdir.mkdir(parents=True, exist_ok=True)
635     write_trajectory_csv(rows, fdir / "trajectory.csv")
636     write_labels_csv(reset_gts, fdir / "labels.csv")
637
638     iou = 0.5
639     prov = {
640         "slug": out_slug,
641         "schema_version": CURRENT_SCHEMA,
642         "kind": "cleared_real",
643         "label": str(label),
644         "license": license,
645         "minors_free": True,
646         "owned": True,
647         "fps": _round(fps),
648         "frame_width": _round(frame_width),
649         "iou": _round(iou),
650         "time_origin_reset": bool(time_origin_reset),
651         "windowing": {
652             "vt": PINNED_VT,
653             "mg": PINNED_MERGE_GAP,
654             "mwd": PINNED_MIN_WINDOW,
655             "max_win": PINNED_MAX_WINDOW,
656         },
657         "paths": {
658             "trajectory": "trajectory.csv",
659             "labels": "labels.csv",
660             "expected": "expected.json",
661         },
662         "gt_hash": gt_hash(reset_gts),
663         "source_note": str(source_note),
664         "note": (
665             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
666             biometric-"
667             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
668             NON-"
669             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
670             'anonymous'. "
671             "Tier-0 / CONTROL-only. $6: behaviour-locking, not correctness; sits behind
672             counsel (P2).",
673         ),
674     }
675     _write_json(fdir / "provenance.json", prov)
676
677     expected = replay_fixture(fdir) # REUSE M1 ? correct-by-construction
678     snapshot
679     _write_json(fdir / "expected.json", expected)
680
681     # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, $4c / ?)
682     -----
683     source_paths = [str(traj_path), str(labels_path)]
684     _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
685                     where=f"{out_slug}/expected.json", source_paths=source_paths)
686     _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
687                     where=f"{out_slug}/provenance.json", source_paths=source_paths)
688
689     # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
690     -----
691     _append_manifest_entry(prov, fixtures_dir)
692     return expected
693
694 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
695     """A manifest row for a single fixture, derived from its provenance block."""
696     slug = prov["slug"]
697     return {

```

```

692         "slug": slug,
693         "fps": _round(float(prov["fps"])),
694         "frame_width": _round(float(prov["frame_width"])),
695         "windowing": dict(prov.get("windowing", {})),
696         "kind": prov.get("kind", "synthetic"),
697         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
698         "paths": {
699             "dir": slug,

```

4. assistant (2026-06-16T06:33:07.301Z)

Let me look at the relevant constants (FORBIDDEN_KEYS, _MD5_RE) and the CLI to verify the finding's claims.

5. user (2026-06-16T06:33:09.402Z)

```

88:FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
95:_MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
504:     Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
511:     for key in FORBIDDEN_KEYS:
517:         m = _MD5_RE.search(text)

```

6. user (2026-06-16T06:33:10.170Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\gf-doc.

7. assistant (2026-06-16T06:33:12.599Z)

The test file path was wrong. Let me find it.

8. user (2026-06-16T06:33:13.175Z)

tests\test_golden_real_fixtures.py

9. user (2026-06-16T06:33:14.222Z)

```

80         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
81     )
82     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
83         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
84     )
85
86     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
87     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
88     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
89
90     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
91     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
92
93     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
94     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
95     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
96
97
98     # =====
99     # Synthetic trajectory generation (deterministic ? NO random)

```

10. assistant (2026-06-16T06:33:17.268Z)

Now let me read the actual test file to verify the claim about test line 155 and how `source\_note` is tested.

11. user (2026-06-16T06:33:17.717Z)

```
1      """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
2      gate.
3      These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
4      an OWNED,
5      minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
6      fixture) and
7      its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
8      Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
9      NO GPU.
10     We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
11     and the
12     ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
13     CSVs, then
14     prove:
15     * the refusal gate blocks every unsafe combination (M2);
16     * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
17     FORBIDDEN_KEYS);
18     * the time-origin reset is metric-INVARIANT (score identical with/without; only
19     absolute
20     ``control_segments`` shift toward 0);
21     * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
22
23     §6 honest-limits caveat: a green suite proves only that the deterministic post-
24     trajectory transforms
25     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
26     correctness.
27     """
28     import pytest
29
30     from backend.eval import golden_fixtures as gf
31
32     # -----
33     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
34     # -----
35
36     def _write_source(tmp_path, *, frame_offset: int = 0):
37         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
38
39         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
40         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
41         absolute
42         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
43         """
44         spec = gf.builtin_specs()[0] # clean_single_rally
45         fps, fw = spec.fps, spec.frame_width
46         rows = gf.make_synthetic_trajectory(spec)
47         if frame_offset:
48             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
49         gts = list(spec.gts)
50         if frame_offset:
51             off_sec = frame_offset / fps
52             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
53
54         traj = tmp_path / "src_trajectory.csv"
55         labels = tmp_path / "src_labels.csv"
56         gf.write_trajectory_csv(rows, traj)
```

```

50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(
60         "kwargs, why",
61         [
62             ({"minors_free": False, "owned": True, "license": "owned"},
63              "minors_free=False"),
64             ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
65             ({"minors_free": True, "owned": True, "license": ""}, "license=' '"),
66             ({"minors_free": True, "owned": True, "license": " "}, "license=blank"),
67             ({"minors_free": True, "owned": True, "license": None}, "license=None"),
68         ],
69     )
70     def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
71         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
72         blank license.
73
74         The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
75         fixture path
76         must be impossible to reach silently.
77         """
78         traj, labels, fps, fw, _ = _write_source(tmp_path)
79         with pytest.raises(ValueError) as ei:
80             gf.freeze_real_fixture(
81                 "scenario", traj, labels,
82                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
83         # RefusalError is a ValueError subclass; the message must name a reason.
84         assert isinstance(ei.value, gf.RefusalError), why
85         assert "REFUSED" in str(ei.value), why
86         # Nothing was written for a refused freeze.
87         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
88         "fixtures").iterdir()), (
89             f"{why}: a refused freeze must not write any fixture")
90
91     # -----
92     # P1 ? de-attribution
93     # -----
94
95     def test_real_fixture_is_deattributed(tmp_path):
96         """A successful safe freeze is de-attributed: kind=cleared_real, fx-prefixed
97         surrogate slug,
98         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
99         fixtures = tmp_path / "fixtures"
100         traj, labels, fps, fw, _ = _write_source(tmp_path)
101         expected = gf.freeze_real_fixture(
102             "clean_rally", traj, labels,
103             license="owned", minors_free=True, owned=True,
104             fps=fps, frame_width=fw, fixtures_dir=fixtures)
105
106         slug = expected["slug"]
107         assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
108         {slug!r}"
109
110         prov = gf._read_json(fixtures / slug / "provenance.json")
111         assert prov["kind"] == "cleared_real"
112         assert prov["minors_free"] is True and prov["owned"] is True

```



```

109     assert prov["license"] == "owned"
110     assert prov["slug"] == slug
111     # No attribution keys / no source filename anywhere in the provenance.
112     for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
113         assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
114     prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
115     for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
116         assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
117
118     # expected.json carries no biometric/identity/non-shuttle stream.
119     exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
120     for forbidden in gf.FORBIDDEN_KEYS:
121         assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
122
123     # The fixture is registered in the manifest with kind=cleared_real.
124     manifest = gf.load_manifest(fixtures)
125     rows = [e for e in manifest if e["slug"] == slug]
126     assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
127
128
129     def test_supplied_slug_is_honored(tmp_path):
130         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
131         fixtures = tmp_path / "fixtures"
132         traj, labels, fps, fw, _ = _write_source(tmp_path)
133         expected = gf.freeze_real_fixture(
134             "scenario", traj, labels, slug="fx_custom00",
135             license="owned", minors_free=True, owned=True,
136             fps=fps, frame_width=fw, fixtures_dir=fixtures)
137         assert expected["slug"] == "fx_custom00"
138         assert (fixtures / "fx_custom00" / "expected.json").is_file()
139
140
141     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
142         """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
143         fixtures = tmp_path / "fixtures"
144         traj, labels, fps, fw, _ = _write_source(tmp_path)
145         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
146                                   license="owned", minors_free=True, owned=True,
147                                   fps=fps, frame_width=fw, fixtures_dir=fixtures)
148         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
149                                   license="owned", minors_free=True, owned=True,
150                                   fps=fps, frame_width=fw, fixtures_dir=fixtures)
151         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
152         assert {e1["slug"], e2["slug"]} <= slugs
153
154
155     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
156         """source_note is non-identifying by contract ? a path/name in it trips the positive
scan."""
157         fixtures = tmp_path / "fixtures"
158         traj, labels, fps, fw, _ = _write_source(tmp_path)
159         with pytest.raises(gf.RefusalError):
160             gf.freeze_real_fixture(
161                 "scenario", traj, labels, slug="fx_leak0001",
162                 license="owned", minors_free=True, owned=True,
163                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
164                 source_note=str(traj)) # leaking the source path ? must be refused
165
166
167     # -----

```

```

168 # P1 (d) ? time-origin reset is metric-invariant
169 # -----
170
171
172 def test_time_origin_reset_is_metric_invariant(tmp_path):
173     """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
frozen
174     WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
timeline)."""
175     offset_frames = 9000 # +5 minutes at 30fps ? a large absolute capture-timeline
offset
176     traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
177
178     with_reset = gf.freeze_real_fixture(
179         "scenario", traj, labels, slug="fx_reset_on",
180         license="owned", minors_free=True, owned=True,
181         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
182         time_origin_reset=True)
183     without_reset = gf.freeze_real_fixture(
184         "scenario", traj, labels, slug="fx_reset_off",
185         license="owned", minors_free=True, owned=True,
186         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
187         time_origin_reset=False)
188
189     # (1) Score is bit-identical ? metrics.score is built from pure differences.
190     assert with_reset["score"] == without_reset["score"], (
191         "time-origin reset must be metric-invariant (score built from pure
differences)")
192
193     # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
194     seg_on = with_reset["control_segments"]
195     seg_off = without_reset["control_segments"]
196     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
197     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198     # The shift equals the offset in seconds (within rounding).
199     shift = seg_off[0][0] - seg_on[0][0]
200     assert abs(shift - offset_frames / fps) < 1e-3, (
201         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202     # Reset trajectory starts at frame 0.
203     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204         .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208 # -----
209 # P1 (e/f) ? round-trip / correct-by-construction
210 # -----
211
212
213 def test_freeze_real_round_trips_check(tmp_path):
214     """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215     returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216     fixtures = tmp_path / "fixtures"
217     traj, labels, fps, fw, _ = _write_source(tmp_path)
218     expected = gf.freeze_real_fixture(
219         "scenario", traj, labels, slug="fx_rt000001",
220         license="owned", minors_free=True, owned=True,
221         fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223     fdir = fixtures / expected["slug"]
224     committed = gf._read_json(fdir / "expected.json")

```

```

225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",
240                 license="owned", minors_free=True, owned=True,
241                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
242

```

12. assistant (2026-06-16T06:33:22.107Z)

The audit finding's evidence is precise. Let me verify the core empirical claim: that a free-text identifier in `source\_note`/`label` passes the scan. The finding says line 155's test only catches a leak because it passes a PATH token. Let me confirm by tracing through `\_scan\_forbidden` with a free-text example, and check the CLI help text claim.

Let me find the CLI argument definitions to verify the \$4c / `--source-note` / `--label` help claim.

13. user (2026-06-16T06:33:24.433Z)

```

570:     source_note: str = "",
663:         "source_note": str(source_note),
925:     ap.add_argument("--labels", help="--freeze-real) source golden labels CSV (start,end)")
934:     ap.add_argument("--label", default="cleared_real", help="--freeze-real) non-identifying
scenario label")
935:     ap.add_argument("--source-note", dest="source_note", default="",
965:                     ("--labels", args.labels),
977:         time_origin_reset=args.time_origin_reset, source_note=args.source_note)

```

14. user (2026-06-16T06:33:27.721Z)

```

920         "--freeze-real (default: a random fx_<hex> id)")
921     ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
922     ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
diff!)")
923     # --freeze-real (P1 / M2 refusal gate) options:
924     ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
925     ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
926     ap.add_argument("--license", dest="license_", default=None,
927                     help="--freeze-real) the source license tag (required, non-blank)")
928     ap.add_argument("--owned", action="store_true",
929                     help="--freeze-real) assert owner-recorded Fork-A source
(required)")
930     ap.add_argument("--minors-free", dest="minors_free", action="store_true",
931                     help="--freeze-real) assert the source contains NO minors
(required)")
932     ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
933     ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-

```

```

real) source frame width (px)")
934     ap.add_argument("--label", default="cleared_real", help="--freeze-real non-
identifying scenario label")
935     ap.add_argument("--source-note", dest="source_note", default="",
936                     help="--freeze-real non-identifying note (NEVER a path/name/id)")
937     ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
938                     help="--freeze-real DISABLE the anti-attribution time-origin reset
(default ON)")
939     ap.set_defaults(time_origin_reset=True)
940     ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941                     help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942     args = ap.parse_args(list(argv) if argv is not None else None)
943
944     fixtures_dir: Path = args.fixtures_dir
945
946     if args.freeze_synthetic:
947         slugs = freeze_synthetic(fixtures_dir)
948         print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
949         print(f"  -> {fixtures_dir}")
950         return 0
951
952     if args.refresh_snapshot:
953         if not args.slug and not args.all:
954             ap.error("--refresh-snapshot requires --slug S or --all")
955         if not args.reason:
956             print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
"
957                 "rubber-stamp a real regression ? review the visible diff ($6).",
file=sys.stderr)
958         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
959         print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}"
+ (f" (reason: {args.reason})" if args.reason else ""))
960         return 0
961
962     if args.freeze_real:
963         missing = [n for n, v in (("--trajectory", args.trajectory),
964                                 ("--labels", args.labels),
965                                 ("--fps", args.fps),
966                                 ("--frame-width", args.frame_width)) if v is None]
967
968         if missing:
969             ap.error("--freeze-real requires " + ", ".join(missing))
970         try:
971             expected = freeze_real_fixture(
972                 args.label, args.trajectory, args.labels,
973                 license=args.license_ if args.license_ is not None else "",
974                 minors_free=args.minors_free, owned=args.owned,
975                 fps=args.fps, frame_width=args.frame_width,
976                 fixtures_dir=fixtures_dir, slug=args.slug,
977                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
978         except RefusalError as e:
979             # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
zero.

```

15. assistant (2026-06-16T06:33:32.429Z)

Now I have confirmed the code. Let me empirically verify the central claim ? that a free-text `source\_note`/`label` passes the scan ? by actually running the function. The dev container lacks numpy/cv2/torch, but `golden\_fixtures.py` may import them transitively. Let me check whether I can run it, or at minimum trace `\_scan\_forbidden` logically with a free-text example.

Let me first check the imports to see if it's runnable here.

16. user (2026-06-16T06:33:33.602Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3      `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and uses **no numpy**.
10
11      The replay reuses the already-shipped eval stack VERBATIM::
12
13          parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14              ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15              ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17      Tier-0 / privacy invariants (D3, $4c, $6 of the doc), enforced here by construction:
18
19      * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20      provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21      * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"`.
22      * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23      person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24      * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25      provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26      ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28      $6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29      suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30      frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31      that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32      field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34      Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-
only
35      for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36      """
37      from __future__ import annotations
38
39      import argparse
40      import json
41      import math
42      import os
43      import re
44      import sys
45      from dataclasses import dataclass
46      from pathlib import Path
47      from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
```

```

50     from backend.eval import distill_local
51     from backend.eval.experiment import CONTROL, VideoCandidates, predict
52     from backend.eval.gt_loader import load_gt_intervals
53     from backend.eval.metrics import score
54     from backend.eval.regression import gt_hash
55     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
56
57     Interval = Tuple[float, float]
58
59     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
60     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
61     CURRENT_SCHEMA = 1
62
63     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
64     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
65     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
66
67     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
68     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
69     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
70     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
71     #: re-states these in its provenance; the replay reads them from there.
72     PINNED_VT = 0.02
73     PINNED_MERGE_GAP = 2.0
74     PINNED_MIN_WINDOW = 2.0
75     PINNED_MAX_WINDOW = 0.0 # 0 = bounded-windowing OFF (W1), matching the SERVED default

```

17. assistant (2026-06-16T06:33:37.263Z)

The module claims to be pure-stdlib on the CONTROL path. Let me try to actually run the empirical test the audit describes ? freezing a fixture with a free-text `source\_note` and `label` and checking whether it gets committed without refusal. This is the definitive verification.

18. user (2026-06-16T06:33:38.469Z)

3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]

19. user (2026-06-16T06:33:50.512Z)

```

FROZE WITH NO REFUSAL ? slug: fx_test0001
label committed: 'mens singles final'
source_note committed: 'Lin Dan vs Lee, India Open 2024, Court 1'

```

20. assistant (2026-06-16T06:33:55.135Z)

The empirical test confirms the audit finding's central claim exactly. A free-text `source\_note='Lin Dan vs Lee, India Open 2024, Court 1'` and `label='mens singles final'` froze with NO refusal and committed both identifying strings verbatim into `provenance.json`.

Now let me verify the finding's secondary claim ? that the existing test at line 155 only catches a leak because it passes a PATH token (`str(traj)`), masking the free-text hole. I've already read the test (lines 155-164): it passes `source\_note=str(traj)`, which is a path token caught by category (c) of `\_scan\_forbidden`. Let me confirm there's no other test covering free-text identifiers.

21. user (2026-06-16T06:33:56.799Z)

```

155: def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
156:     """source_note is non-identifying by contract ? a path/name in it trips the positive
scan."""
164:         source_note=str(traj)) # leaking the source path ? must be refused

```

22. assistant (2026-06-16T06:34:01.787Z)

Confirmed. The only `source\_note` test (line 155) passes `str(traj)` ? a path token caught by category (c). The test's own docstring claims "source_note is non-identifying by contract ? a path/name in it trips the positive scan," but the empirical run proves a *free-text* identifier (a person's name / tournament / court ? not a path) does NOT trip the scan.

Let me also verify the doc's `§4c` claims to assess whether the finding's "doc honesty" angle holds. Let me check what the doc says about this path.

23. user (2026-06-16T06:34:07.289Z)

(Bash completed with no output)

24. user (2026-06-16T06:34:12.005Z)

```

1: # Golden Regression Fixtures ? video-free, non-attributable regression suite
34: **Goal:** commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture
corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression
signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.
70: [Omitted long matching line]
121: | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-name/non-
surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |
144: | **P0** | **Leak remediation + required CI leak-scan.** Scrub real names/paths in
`eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`,
`backend/eval/eval_partial_labels.py`, `tests/test_cleanup_caches.py` ? opaque surrogate keys +
private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a
**required status check** ; decide on git history (PR #77, accept vs `filter-repo`). | ? | No
(owned data hygiene) | ? | ? |
157: [Omitted long matching line]
189: - [ ] The **CI leak-scan is a required status check** and passes (no MD5 / real-name / non-
surrogate / plaintext-SOPS in the tree).

```

25. assistant (2026-06-16T06:34:15.098Z)

Let me look at section 4c of the doc specifically, which the code comments reference, to assess the doc-honesty angle.

26. user (2026-06-16T06:34:18.058Z)

```

22: - ? **Determinism trap** *(verified, corroborated by two independent red-teams)*: the
learned variants (`shuttle_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC_ARCH` logistic reg
46: | D5 | **Cross-machine gate is CONTROL/static-only** *(design)* | both red-teams [verified
cause] | Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with toler
91: ### 4c. Anti-attribution measures (Tier-0)
92: - **Opaque RANDOM surrogate id** (`os.urandom` + a private surrogate?source map kept off-
git) ? *not* the MD5 `video_id`, and *not* HMAC(salt, video_id?name) (red-team: brute-force
97: - **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue quasi-
identifier.
108: **Default = out-of-repo + token** (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero
sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS
121: | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-name/non-
surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |
144: | **P0** | **Leak remediation + required CI leak-scan.** Scrub real names/paths in
`eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`,
`backend/eval/eval_par
147: | **M3** | **Characterization gate** ? `tests/test_golden_fixtures.py`: parse-compared
(exact-int / approx-float 1e-6), **CONTROL-only**, positive no-person/audio assertion, pertur
150: | **P1** | **De-attribution + minimization** ? `golden_fixtures.freeze_real_fixture`

```

(extends the M1 module, not a separate script): ****opaque random `fx\_hex` surrogate slug****,
157: ****Must-fix-before-any-public-commit (red-team), folded into the rows above:**** ? numeric-
tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in
189: - [] The ****CI leak-scan is a required status check**** and passes (no MD5 / real-name / non-
surrogate / plaintext-SOPS in the tree).
208: - ****2026-06-16**** ? Initial draft synthesized from two multi-agent workflows (mechanics
design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verifie

27. user (2026-06-16T06:34:22.418Z)

```
91     ### 4c. Anti-attribution measures (Tier-0)
92     - **Opaque RANDOM surrogate id** (`os.urandom` + a private surrogate?source map kept
off-git) ? *not* the MD5 `video_id`, and *not* HMAC(salt, video_id?name) (red-team: brute-
forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source
filename).
93     - **Metadata strip** ? no filename, video_id, match/event/player identity, capture date.
94     - **Time-origin reset** ? subtract per-fixture `t0` from all start/end/gts. **Provably
metric-invariant** (`metrics.interval_iou` and start/end-error are pure differences) ? bit-
identical scores, severed absolute timeline.
95     - **Stream minimization** ? Tier-0 keeps shuttle block only; person/audio excluded;
pose/gait/face have **no field in the schema by design**.
96     - **Raw-trajectory exclusion** ? never commit the per-frame `(Frame,Visibility,X,Y)`
CSV.
97     - **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue
quasi-identifier.
98     - **Provenance hard-fail gate** ? the de-attribution pass refuses to emit a public
fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.
99
100    ### 4d. The two gates
101    - **Characterization/snapshot** ? replay fixture, **parse-compare** JSON (never byte-
compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields,
`approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints
(incommensurable ? loud fail). Catches any behaviour change even at constant F1. **CONTROL path
only** across machines.
102    - **Quality-floor** ? score vs committed labels; assert per-video + mean F1 ? floor in
`eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so
a served-default change is flagged incommensurable, not silently compared); emit `eval_stats`
paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta.
Synthetic-tier floors measure **correctness, not field quality** (documented).
103
104    ### 4e. Serialization & optional protobuf
105    JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is
optional (01)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3
`rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`).
Protobuf is **encoding, not secrecy** (trivially `--decode_raw`).
106
107    ### 4f. Encryption & keys (Tier-1 only)
108    **Default = out-of-repo + token** (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero
sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS
+ age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting
in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI
secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The
**HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus,
rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.
109
110    ---
111
112    ## 5. Threat model (who's protected vs not)
```

28. assistant (2026-06-16T06:34:27.220Z)

Now I have a complete picture. The doc §4c "Metadata strip" bullet (line 93) states: "no filename, video_id, match/event/player identity, capture date." But the code commits `label` and `source\_note` verbatim ? fields that can carry exactly "match/event/player identity." The doc describes the de-attribution goal, but the code does NOT enforce the strip for these two

operator-supplied free-text fields, and there's no caveat that they're committed unscanned.

Let me verify there isn't a separate validation/sanitization of `label`/`source\_note` elsewhere that I may have missed.

29. user (2026-06-16T06:34:29.167Z)

```
137:  "labels.csv". For a synthetic fixture they line up with the rallies by construction,
202: def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
203:  """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
300:  ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
312:  labels_csv = fdir / "labels.csv"
328:  gts = load_gt_intervals(str(labels_csv), strict=True)
394:  "labels": "labels.csv",
407:  ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
410:  + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
413:  traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
414:  if not traj_csv.is_file() or not labels_csv.is_file():
416:      f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to generate
them")
430:  """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
440:      write_labels_csv(spec.gts, fdir / "labels.csv")
449:  """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
450:  NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real regression;
the
465: # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
494: #: false-positive against the literal ``trajectory.csv``/``labels.csv``/``expected.json``.
496:  "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
552: def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
553:  """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
558:  label: str,
560:  labels_csv_path: Any,
570:  source_note: str = "",
572:  """Turn an OWNED, minors-free real trajectory + golden labels into a committable, de-
attributed
579:  reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
608:  labels_path = Path(str(labels_csv_path))
613:  # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
617:  gts = load_gt_intervals(str(labels_path), strict=True)
620:  # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
630:  reset_gts = _reset_labels(gts, t0_sec)
636:  write_labels_csv(reset_gts, fdir / "labels.csv")
643:  "label": str(label),
659:  "labels": "labels.csv",
663:  "source_note": str(source_note),
677:  source_paths = [str(traj_path), str(labels_path)]
701:  "labels": f"{slug}/labels.csv",
736:  "labels": f"{spec.slug}/labels.csv",
910:  help="generate synthetic trajectory+labels for all built-in scenarios,
then "
913:  help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
917:  help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
925:  ap.add_argument("--labels", help="--freeze-real) source golden labels CSV (start,end)")
934:  ap.add_argument("--label", default="cleared_real", help="--freeze-real) non-identifying
scenario label")
```

```

935:     ap.add_argument("--source-note", dest="source_note", default="",
965:                     ("--labels", args.labels),
972:                     args.label, args.trajectory, args.labels,
977:                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)

```

30. assistant (2026-06-16T06:34:45.835Z)

Confirmed ? there is no sanitization, allow-list, or controlled-vocabulary check on `label` or `source\_note` anywhere. They flow `str(...)` ? `provenance.json` ? scanned by `\_scan\_forbidden`, which only catches biometric substrings, a 32-hex MD5, or source-path tokens. A free-text identifier matches none of these.

I have fully verified the finding. Let me record my verdict.

The finding is **CONFIRMED** on every checkable claim:

1. **`\_scan\_forbidden` scans only 3 categories** ? `golden\_fixtures.py:501-535`: (a) `FORBIDDEN\_KEYS` biometric substrings (`golden\_fixtures.py:88` = `"person","pose","gait","face","audio","player"`), (b) a 32-hex MD5 via `\_MD5\_RE` (`:95`), (c) source-PATH tokens (`sp`/`p.name`/`p.stem`, `:522-535`). None matches a free-text identifier.
2. **`label` (`:643`, default `cleared\_real`) and `source\_note` (`:663`) are written verbatim** into `provenance.json` via `str(...)` , then scanned at `:680-681`. No allow-list/charset/vocabulary check exists anywhere (grep confirms).
3. **Empirically proven** ? running `freeze\_real\_fixture(... label='mens singles final', source\_note='Lin Dan vs Lee, India Open 2024, Court 1')` FROZE with NO refusal and committed both strings verbatim into `provenance.json` (output: `FROZE WITH NO REFUSAL`; `label` committed: `mens singles final`; `source\_note` committed: `Lin Dan vs Lee, India Open 2024, Court 1`).
4. **The only `source\_note` test masks the hole** ? `tests/test\_golden\_real\_fixtures.py:155-164` passes `source\_note=str(traj)` (a PATH token caught by category (c)). Its docstring even claims "source_note is non-identifying by contract ? a path/name in it trips the positive scan," which is false for a non-path free-text name. No test exercises a free-text identifier.
5. **The doc claim is dishonest by omission** ? `docs/GOLDEN\_REGRESSION\_FIXTURES.md:93` ("Metadata strip ? no filename, video_id, match/event/player identity, capture date") and the CLI help `--source-note ... NEVER a path/name/id` (`golden\_fixtures.py:936`) assert the contract but nothing enforces it for names/ids in these two fields.

This is a real privacy/de-attribution hole in a path explicitly designed to commit de-attributed data publicly. The operator can leak player/match/event identity into a committed `provenance.json` while the "positive privacy assertion" passes ? and the test suite gives false confidence by only testing the path-token case. **isMustFix = true**, because the suite/tooling is the safeguard for committing real data publicly, and relying on it as-is permits exactly the leak it claims to prevent.

31. user (2026-06-16T06:34:55.883Z)

Structured output provided successfully

32. assistant (2026-06-16T06:35:03.526Z)

Verdict: **CONFIRMED** (isMustFix = true).

The audit finding holds on every checkable claim, and I proved the central one empirically:

- `\_scan\_forbidden` (`backend/eval/golden\_fixtures.py:501-535`) scans for only three things ? biometric substrings (`FORBIDDEN\_KEYS`, `:88`), a 32-hex MD5 (`\_MD5\_RE`, `:95`/`:517`), and source-path tokens (`:522-535`). A free-text identifier matches none.
- `label` (`:643`) and `source\_note` (`:663`) are committed verbatim into `provenance.json` with no allow-list/charset/vocabulary check anywhere.
- Empirical run: `freeze\_real\_fixture(... label='mens singles final', source\_note='Lin Dan vs Lee, India Open 2024, Court 1')` froze with NO refusal and committed both strings verbatim.

- The only `source\_note` test (`tests/test\_golden\_real\_fixtures.py:155-164`) passes `str(traj)` ? a path token caught by category (c) ? masking the free-text hole; its docstring's "non-identifying by contract" claim is unenforced.
- `docs/GOLDEN\_REGRESSION\_FIXTURES.md:93` and the CLI help (`golden\_fixtures.py:936`, "NEVER a path/name/id") assert a contract that nothing enforces for names/ids.

This is a real de-attribution hole in the exact path designed to commit cleared real data publicly, so it must be fixed before relying on the suite as a privacy safeguard.